

P0401R4

Providing size feedback in the Allocator interface

Published Proposal, 2020-11-14

This version:

<https://wg21.link/P0901R4>

Authors:

[Jonathan Wakely](#)

[Chris Kennelly](#) (Google)

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Utilize size feedback from Allocator to reduce spurious reallocations

Table of Contents

1	Introduction
2	Motivation
2.1	Why not <code>realloc</code>
2.2	Why not ask the allocator for the size
3	Proposal
4	Design Considerations
4.1	<code>allocate</code> selection
4.2	Size Return Value
4.3	<code>deallocate</code> changes
4.4	Interaction with <code>polymorphic_allocator</code>
4.5	Zero-Sized Requests
5	Revision History
5.1	R3 → R4
5.2	R2 → R3
5.3	R1 → R2
	References
	Informative References

§ 1. Introduction

This is a library paper to describe how size feedback can be used with allocators:

- In the case of `std::allocator`, the language feature proposed in [\[P0901R5\]](#) could be used.
- In the case of custom allocators, the availability of the traits interface allows standard library containers to leverage this functionality.

§ 2. Motivation

Consider code adding elements to `vector`:

```
std::vector<int> v = {1, 2, 3};
// Expected: v.capacity() == 3

// Add an additional element, triggering a reallocation.
v.push_back(4);
```

Many allocators only allocate in fixed-sized chunks of memory, rounding up requests. Our underlying heap allocator received a request for 12 bytes ($3 * \text{sizeof}(\text{int})$) while constructing `v`. For several implementations, this request is turned into a 16 byte region.

§ 2.1. Why not `realloc`

`realloc` poses several problems:

- We have unclear strategies for growing a container. Consider the two options (a and b) outlined in the comments of the example below. Assume, as with the [§ 2 Motivation](#) example, that there is a 16 byte size class for our underlying allocator.

```
std::vector<int> v = {1, 2, 3};
// Expected: v.capacity() == 3

// Add an additional element. Does the vector:
// a) realloc(sizeof(int) * 4), adding a single element
// b) realloc(sizeof(int) * 6), attempt to double, needing reallocation
v.push_back(4);

// Add another element.
// a) realloc(sizeof(int) * 5), fails in-place, requires reallocation
// b) size + 1 <= capacity, no reallocation required
v.push_back(5);
```

Option **a** requires every append (after the initial allocation) try to `realloc`, requiring an interaction with the allocator each time.

Option **b** requires guesswork to probe the allocator's true size. By doubling (the typical growth pattern for `std::vector` in `libc++` and `libstdc++`), we skip past the existing allocation's size boundary. While we can use a reduced growth rate, doing so requires more interactions with the allocator (calls to `realloc`), as well as copies (when we cannot resize in-place).

Both of these options add extra branching on the control flow for growing the buffer.

- When the allocator cannot resize the buffer in-place, it allocates new memory and `memcpy`'s the contents. For non-trivially copyable types, this is disastrous. The *language* lacks support for reallocating in-place *without* moving. While this function could be implemented as magic library function, it could not interact with many underlying `malloc` implementations (`libc`, in the case of `libc++` and `libstdc++`) used to build `operator new`, nor could it interact with user-provided replacement functions.

In-place reallocation has been considered in [\[N3495\]](#) (throwing `std::bad_alloc` when unable) and [\[P0894R1\]](#) (returning `false` when unable). This has sometimes been referred to as an "extend" or an "expand" interface. Extending an existing allocation requires a round-trip to the allocator (and its requisite branch/call overheads) to determine whether the allocation *could* be expanded. As discussed previously, we need to probe for the true size of the backing object (until we find extend fails) until we give up and apply a multiplicative expansion factor.

For fixed-size allocators it makes more sense, and is much simpler for containers to adapt to, if the allocator is able to over-allocate on the initial request and inform the caller how much memory was made available.

§ 2.2. Why not ask the allocator for the size

We could also explore APIs that answer the question: "If I ask for `N` bytes, how many do I actually get?" [\[jemalloc\]](#) and [\[TCMalloc\]](#) call this `mallocx`.

- `mallocx` requires an extra call / size calculation. This requires we duplicate work, as the size calculation is performed as part of allocation itself.
- `mallocx` impairs telemetry for monitoring allocation request sizes. If a `mallocx` return value is cached, the user appears to be asking for exactly as many bytes as `mallocx` said they would get.

See also [\[P0901R5\]](#)'s discussion "`mallocx`: not as awesome as it looks."

§ 3. Proposal

Wording relative to [\[N4868\]](#).

We propose a free function to return the allocation size simultaneously with an allocation.

Amend `[memory.syn]`:

```
namespace std {
    // ... elided ...

    // [allocator.traits], allocator traits
    template<class Alloc> struct allocator_traits;

    template<typename Pointer>
    struct allocation_result {
        Pointer ptr;
        size_t count;
    };

    template<typename Allocator>
    constexpr allocation_result<typename Allocator::pointer> allocate_at_least(
        Allocator& a, size_t n);

    // [default.allocator], the default allocator
    template<class T> class allocator;
    template<class T, class U>
        constexpr bool operator==(const allocator<T>&, const allocator<U>&) noexcept;

    // ... elided ...
}
```

Amend `[allocator.traits]`:

```

namespace std {
    template<class Alloc> struct allocator_traits {
        using allocator_type      = Alloc;

        using value_type          = typename Alloc::value_type;

        using pointer              = see below;
        using const_pointer        = see below;
        using void_pointer         = see below;
        using const_void_pointer   = see below;

        using difference_type      = see below;
        using size_type            = see below;

        using propagate_on_container_copy_assignment = see below;
        using propagate_on_container_move_assignment = see below;
        using propagate_on_container_swap            = see below;
        using is_always_equal                        = see below;

        template<class T> using rebind_alloc = see below;
        template<class T> using rebind_traits = allocator_traits<rebind_alloc<T>>;

        [[nodiscard]] static constexpr pointer allocate(Alloc& a, size_type n);
        [[nodiscard]] static constexpr pointer allocate(Alloc& a, size_type n,
                                                         const_void_pointer hint);

        static constexpr void deallocate(Alloc& a, pointer p, size_type n);

        template<class T, class... Args>
            static constexpr void construct(Alloc& a, T* p, Args&&... args);

        template<class T>
            static constexpr void destroy(Alloc& a, T* p);

        static constexpr size_type max_size(const Alloc& a) noexcept;

        static constexpr Alloc select_on_container_copy_construction(const Alloc& rhs);
    };

    template<typename Pointer>
    struct allocation_result {
        Pointer ptr;
        size_t count;
    };

    template<typename Allocator>
    constexpr allocation_result<typename Allocator::pointer> allocate_at_least(
        Allocator& a, size_t n);
}

```

Amend [allocator.traits.types]:

The class template `allocation_result` has the template parameters, data members, and special members specified above. It has no base classes or members other than those specified.

Amend [allocator.traits.members]:

```
[[nodiscard]] static constexpr pointer allocate(Alloc& a, size_type n);
```

- *Returns:* a.allocate(n).

```
[[nodiscard]] static constexpr pointer allocate(Alloc& a, size_type n, const_void_pointer hint);
```

- *Returns:* a.allocate(n, hint) if that expression is well-formed; otherwise, a.allocate(n).

```
template<typename Allocator>  
constexpr allocation_result<typename Allocator::pointer> allocate_at_least(  
    Allocator& a, size_t n);
```

- *Returns:* a.allocate_at_least(n) if that expression is well-formed; otherwise, a.allocate(n).

```
static constexpr void deallocate(Alloc& a, pointer p, size_type n);
```

- *Effects:* Calls a.deallocate(p, n).
- *Throws:* Nothing.

Amend [default.allocator.general]:

```
namespace std {  
    template<class T> class allocator {  
    public:  
        using value_type           = T;  
        using size_type            = size_t;  
        using difference_type      = ptrdiff_t;  
        using propagate_on_container_move_assignment = true_type;  
        using is_always_equal      = true_type;  
  
        constexpr allocator() noexcept;  
        constexpr allocator(const allocator&) noexcept;  
        template<class U> constexpr allocator(const allocator<U>&) noexcept;  
        constexpr ~allocator();  
        constexpr allocator& operator=(const allocator&) = default;  
  
        [[nodiscard]] constexpr T* allocate(size_t n);  
        [[nodiscard]] constexpr allocation_result<T*> allocate_at_least(size_t n);  
        constexpr void deallocate(T* p, size_t n);  
    };  
}
```

Amend [allocator.members]:

Except for the destructor, member functions of the default allocator shall not introduce data races as a result of concurrent calls to those member functions from different threads. Calls to these functions that allocate or deallocate a particular unit of storage shall occur in a single total order, and each such deallocation call shall happen before the next allocation (if any) in this order.

```
[[nodiscard]] constexpr T* allocate(size_t n);
```

- *Mandates:* T is not an incomplete type ([basic.types]).
- *Returns:* A pointer to the initial element of an array of n T.
- *Throws:* bad_array_new_length if `numeric_limits<size_t>::max() / sizeof(T) < n`, or bad_alloc if the storage cannot be obtained.
- *Remarks:* The storage for the array is obtained by calling `::operator new`, but it is unspecified when or how often this function is called. This function starts the lifetime of the array object, but not that of any of the array elements.

```
[[nodiscard]] constexpr allocation_result<T*> allocate_at_least(size_t n);
```

- *Mandates:* T is not an incomplete type ([basic.types]).
- *Returns:* `allocation_result<T*>{ptr, count}`, where ptr is a pointer to the initial element of an array of count T and `count >= n`.
- *Throws:* bad_array_new_length if `numeric_limits<size_t>::max() / sizeof(T) < n`, or bad_alloc if the storage cannot be obtained.
- *Remarks:* The storage for the array is obtained by calling `::operator new`, but it is unspecified when or how often this function is called. This function starts the lifetime of the array object, but not that of any of the array elements.

```
constexpr void deallocate(T* p, size_t n);
```

- *Preconditions:* p is a pointer value obtained from `allocate()`; n equals the value passed as the first argument to the invocation of `allocate` which returned p.
- *Preconditions:* If memory was obtained by a call to `auto [p, count] = a.allocate_at_least(req)`, n shall have a value such that `req <= n <= count`. Otherwise, p shall be the value returned by an earlier call to `allocate` and n shall match the value passed to `allocate`. In either case, p shall not have been invalidated by an intervening call to `deallocate`.
- *Effects:* Deallocates the storage referenced by p.
- *Remarks:* Uses `::operator delete`, but it is unspecified when this function is called.

Amend [allocator.requirements.general], [tab:cpp17.allocator]

<code>a.allocate_at_least(n)</code>	<code>allocation_result<X::pointer>{ptr, count}</code>	Memory is allocated for an array of count T and such an object is created but array elements are not constructed, such that count $\geq n$. <code>allocate_at_least</code> may throw an appropriate exception. See Note C, below.
<code>a.deallocate(p, n)</code>	(not used)	<i>Preconditions:</i> <code>p</code> is a value returned by an earlier call to <code>allocate</code> that has not been invalidated by an intervening call to <code>deallocate</code> . <code>n</code> matches the value passed to <code>allocate</code> to obtain this memory. <i>Preconditions:</i> If memory was obtained by a call to <code>auto [p, count] = a.allocate_at_least(req), n</code> shall have a value such that <code>req ≤ n ≤ count</code> . Otherwise, <code>p</code> shall be the value returned by an earlier call to <code>allocate</code> and <code>n</code> shall match the value passed to <code>allocate</code> . In either case, <code>p</code> shall not have been invalidated by an intervening call to <code>deallocate</code> . <i>Throws:</i> Nothing.

... elided ...

Note C: If `n == 0`, the return value is unspecified. An allocator need not support `allocate_at_least`, but no default is provided in `allocator_traits`. Instead, the function `std::allocate_at_least(a, n)` will default to returning `allocation_result<typename X::pointer>{ a.allocate(n), n }` if `a.allocate_at_least(n)` is not a valid expression.

Amend [version.syn]

```
#define __cpp_lib_allocate_at_least PLACEHOLDER // also in <memory>
```

§ 4. Design Considerations

§ 4.1. `allocate` selection

There are multiple approaches here:

- Return a pointer-size pair, as presented.
- Overload `allocate` and return via a reference parameter. This potentially hampers optimization depending on ABI, as the value is returned via memory rather than a register.

§ 4.2. Size Return Value

In [Prague], LEWG discussed the return value. For compatibility with the existing allocator APIs, which work in units of *objects* rather than bytes, this proposal chooses to continue to return an integer number of objects.

Additionally, for types with non-trivial alignment requirements, we must allocate storage for objects, rather than bytes, as raw bytes do not convey the appropriate alignment needs to the allocator.

For example: In the `std::vector<T>` case, many implementations use 3 pointers for representing the state of the container (begin, end, and capacity). If we preserved the precise value returned by the underlying allocator, we may not be able to legally form the capacity pointer. For these implementations, replacing the capacity pointer with a capacity in bytes would be an ABI break.

§ 4.3. `deallocate` changes

We now require flexibility on the size we pass to `deallocate`. For container types using this allocator interface, they are faced with the choice of storing *both* the original size request as well as the provided size (requiring additional auxillary storage), or being unable to reproduce one during the call to `deallocate`.

As the true size is expected to be useful for the `capacity` of a `string` or `vector` instance, the returned size is available without additional storage requirements. The original (requested) size is unavailable, so we relax the `deallocate` size parameter requirements for these allocations.

In [P0901R5], we anticipate use cases where the user does not store enough information about the size to return a precise, byte-level count. We may be allocating `T`'s and receive enough storage to allocate 3.5 objects. Requiring callers return the precise size may be impractical (`std::vector` frequently stores its capacity as a `T*`) without additionally auxillary storage.

In this paper, we may be similarly unable to return the true size back. In the [Prague] review of this paper, we discussed cases where we rounded down to the multiple of the size being allocated. Alternatively, custom data structures may use `allocator` to obtain storage, but quantize that. In Abseil's [Cord], data is stored in chunks with a length-encoding tag at the start of each chunk. These tags quantize the capacity in `AllocatedSizeToTag` that can be represented to multiples of 16- and 32-bytes. This makes the encoded length more compact, but means we cannot reproduce the size the allocator provided us precisely.

§ 4.4. Interaction with `polymorphic_allocator`

`std::pmr::memory_resource` is implemented using virtual functions. Adding new methods, such as the proposed `allocate` API would require taking an ABI break.

§ 4.5. Zero-Sized Requests

In Prague, LEWG discussed the behavior of `allocate_at_least(allocator, 0)`. This maximizes implementation freedom.

- The return value may not be dereferenceable (even if not `nullptr`).
- Other implementations, such as a stack allocator, could just return the current offset.

§ 5. Revision History

§ 5.1. R3 → R4

- Applied wording feedback from Pablo Halpern and David Stone
- Added discussion about changes to deallocation
- Proposed feature test macro

5.2. R2 → R3

Applied LEWG feedback from [\[Prague\]](#).

- Renamed `sized_ptr_t` to `allocation_result`, based on discussion in Prague that this library proposal (which returns a *count* of storage for objects) have a distinct type from the underlying language proposal ([\[P0901R5\]](#), which returns bytes). This was one of the names bikeshedded during the review of P0901R5 ([\[P0901R5-Review\]](#)).
- Renamed `n` (the returned size) to `count`, based on discussion in Prague to clarify that is measured in objects, rather than bytes. Additionally, augmented discussion of the `deallocate` changes for why implementations may not be able to preserve the precise size returned by the underlying allocator.

Poll: We prefer to return number of bytes.

SF	F	N	A	SA
0	0	6	6	5

- Changed the template parameter to pointer type to allow for fancy pointers.

POLL: Change the template parameter to `sized_ptr_t` to pointer type (to support fancy ptr).

Unanimous consent

- Added design intent and wording that `allocation_result` be compatible with structured bindings. This wording was adapted from the proposed resolution of [\[LWG3373\]](#).
- Added `constexpr`
- Removed default template parameter from `allocation_result`
- Based on LEWG feedback, changed `allocate_at_least` to deduce return type from the allocator. Since the pointer type is chosen by the allocator, the underlying object type is less relevant. This also removed the default template parameter.

POLL: We want to deduce the object type from the allocator (`alloc_least_n [sic]`).

SF	F	N	A	SA
4	8	1	0	0

POLL: We like the design, please revise and do initial wording review (with Pablo, and David Stone) send to LWG (tentatively Ready).

SF	F	N	A	SA
6	12	0	0	0

5.3. R1 → R2

Applied LEWG feedback from [\[Cologne\]](#).

Poll: We want a feature like this.

SF	F	N	A	SA
2	9	2	0	0

- As users may specialize `std::allocator_traits`, this revision moves to a free function `std::allocate_at_least` as suggested via a LEWG poll.

Poll: Name choice

5	allocate_with_size
4	overallocate
2	allocate_with_oversubscribe
6	allocate_oversize
14	allocate_at_least

Poll: Prefer a struct rather than a requirement of structured bindings.

SF	F	N	A	SA
2	8	0	2	1

§ References

§ Informative References

[Cologne]

Cologne Meeting Minutes. 2019-07-17. URL: <http://wiki.edg.com/bin/view/Wg21cologne2019/P0401>

[Cord]

absl::Cord. URL: <https://github.com/abseil/abseil-cpp/blob/master/absl/strings/cord.h>

[JEMALLOC]

jemalloc(3) - Linux man page. URL: <http://jemalloc.net/jemalloc.3.html>

[LWG3373]

{to,from}_chars_result and format_to_n_result need the 'we really mean what we say' wording. URL: <https://cplusplus.github.io/LWG/lwg-active.html#3373>

[N3495]

Ariane van der Steldt. inplace realloc. 7 December 2012. URL: <https://wg21.link/n3495>

[N4868]

Working Draft, Standard for Programming Language C++. 2019-10-18. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/n4868.pdf>

[P0894R1]

realloc for C++. 2019-01-18. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0894r1.md>

[P0901R5]

Size feedback in operator new. 2019-10-03. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0901r5.html>

[P0901R5-Review]

P0901R5 LEWG Review Prague. 2020-02-10. URL: <http://wiki.edg.com/bin/view/Wg21prague/P0901>

[Prague]

P0401R2 LEWG Review Prague. 2020-02-14. URL: <http://wiki.edg.com/bin/view/Wg21prague/P0401>

[TCMalloc]

TCMalloc. URL: <https://github.com/google/tcmalloc>